



School: Campus:

Academic Year: Subject Name: Subject Code:

Semester: Program: Branch: Specialization:

Date:

Applied and Action Learning

(Learning by Doing and Discovery)

Name of the Experiment: ERC-20 Basics – Tokenization Concepts

***Coding Phase: Pseudo Code / Flow Chart / Algorithm**

1. Launch **Remix IDE** in your browser.
2. Inside the **contracts** directory, create a new Solidity file named **MyToken.sol**.
3. Write the **ERC-20 smart contract code** within this file.
4. Compile the contract using the Solidity compiler to ensure there are no errors.
5. After successful compilation, select **Injected Provider – MetaMask** as the deployment environment.
6. Deploy the smart contract by confirming the **transaction request** through your MetaMask wallet.
7. Once deployed, copy the **contract address** displayed under the **Deployed Contracts** section.
8. Open your **MetaMask wallet**, navigate to the **Tokens** tab, and choose to **import a custom token**.
9. Make sure you are connected to the **Sepolia Test Network**, then paste the copied **contract address** into the respective field and click **Next**.
10. Click on **Import**, and your custom ERC-20 token will be successfully added to your wallet.
11. Return to the **Tokens** section and select your newly imported token.
12. Click on **Send**, enter the recipient's **public address or ENS domain**, and proceed by clicking **Continue**.
13. Confirm the transaction, and your token will be transferred successfully.

***Software used:**

- Laptop
- Web Browser
- Remix IDE
- MetaMask Wallet
- Sepolia faucet

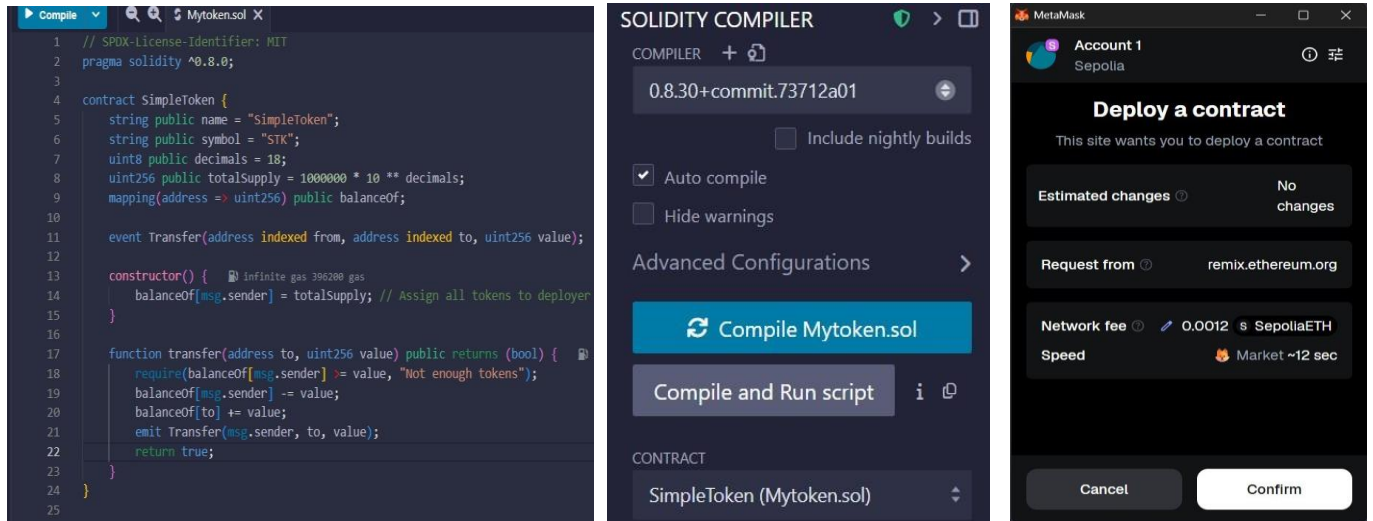
Page No.....

*** As applicable according to the experiment.
Two sheets per experiment (10-20) to be used.**

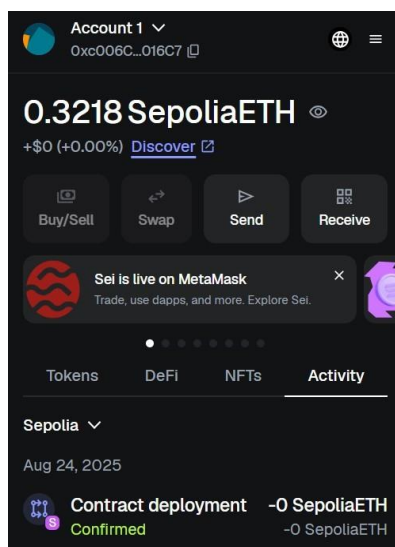
* Testing Phase: Compilation of Code (error detection)

NO ERROR

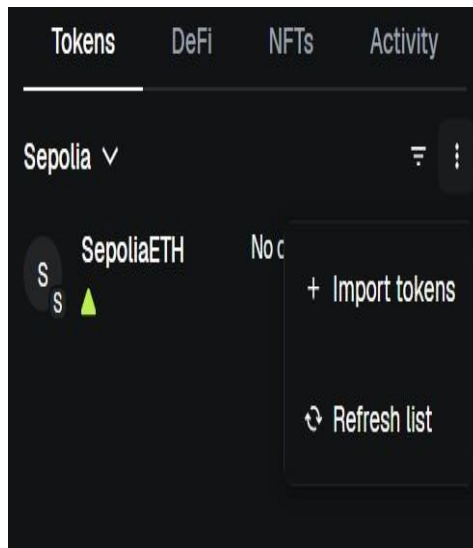
* Implementation Phase: Final Output (no error)



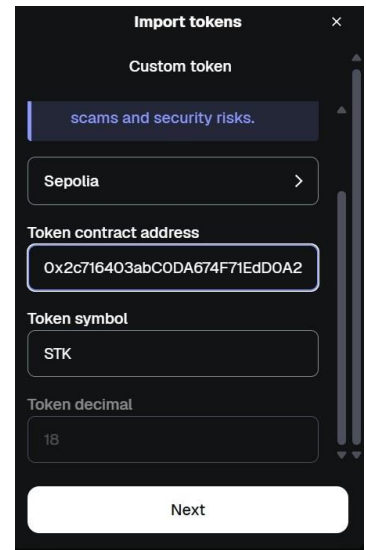
The top row shows the development environment. On the left, a code editor displays Solidity code for a contract named `SimpleToken`. The code includes a constructor that assigns all tokens to the deployer and a `transfer` function. In the center, the **SOLIDITY COMPILER** interface shows the file `Mytoken.sol` being compiled. The `COMPILE` button is highlighted. On the right, the **MetaMask** interface shows the **Deploy a contract** screen. The network is set to **Sepolia**, and the **Confirm** button is visible.



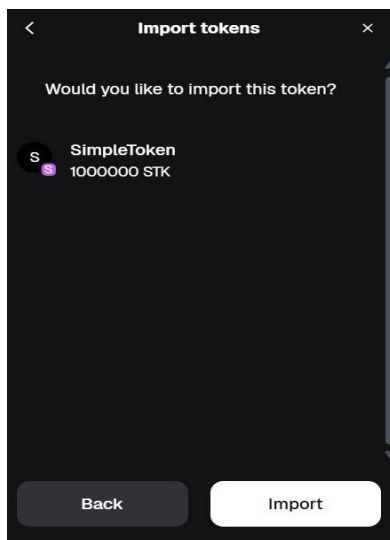
This screenshot shows the MetaMask account overview for **Account 1** on the **Sepolia** network. The balance is **0.3218 SepoliaETH**. Below the balance, there are buttons for **Buy/Sell**, **Swap**, **Send**, and **Receive**. A notification banner states "Sei is live on MetaMask". At the bottom, there is a section for **Contract deployment** showing a confirmed transaction.



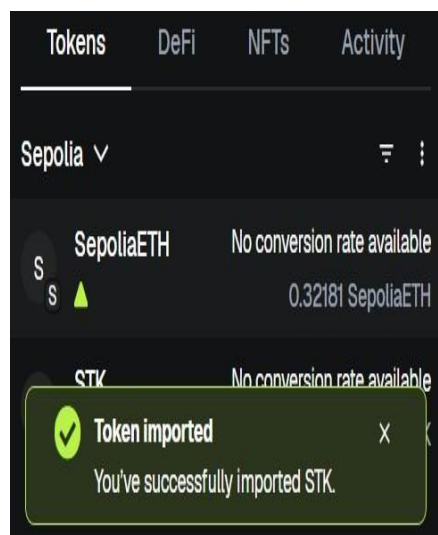
The screenshot shows the **Tokens** tab in MetaMask. It displays the **Sepolia** network and a list of tokens. The **SepoliaETH** token is listed with a green upward arrow. A modal overlay is visible with the text **+ Import tokens** and a **Refresh list** button.



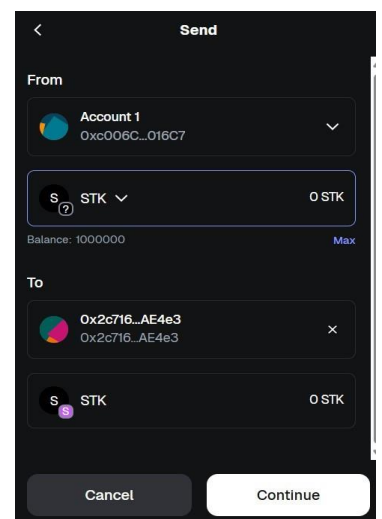
This screenshot shows the **Import tokens** screen in MetaMask. It prompts the user to enter the **Token contract address** (`0x2c716403abC0DA674F71EdD0A2`), the **Token symbol** (`STK`), and the **Token decimal** (`18`). A **Next** button is at the bottom.



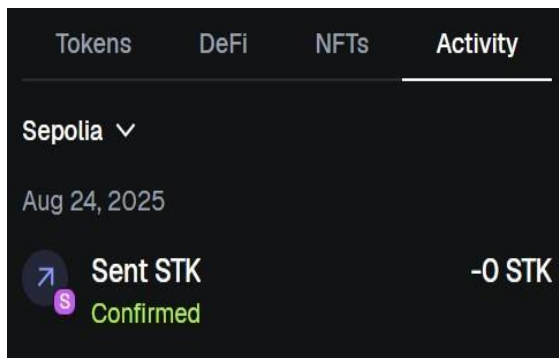
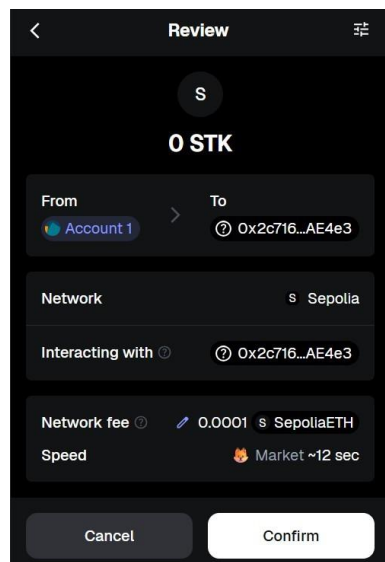
This screenshot shows the confirmation screen for importing a token. It asks "Would you like to import this token?" and displays the token details: **SimpleToken** with a balance of **1000000 STK**. There are **Back** and **Import** buttons at the bottom.



The screenshot shows the **Tokens** tab after the token has been imported. It now lists **SepoliaETH** and **STK**. A green notification bubble at the bottom says **Token imported** and "You've successfully imported STK."



This screenshot shows the **Send** screen in MetaMask. The **From** account is **Account 1** on the **Sepolia** network. The **To** field shows the address `0x2c716...AE4e3`. The token selected is **STK** with a balance of **0 STK**. There are **Cancel** and **Continue** buttons at the bottom.



*Observation:

- **ERC-20** establishes a **standard interface for fungible tokens** on the Ethereum blockchain, ensuring seamless compatibility across wallets, exchanges, and decentralized applications (dApps).
- Every **ERC-20 token** implements essential functions such as **transfer**, **approve**, and **transferFrom**, enabling secure, consistent, and standardized token operations.
- **Tokenization** converts both real-world and digital assets into **ERC-20 tokens**, allowing easy ownership transfer, trading, and management on the blockchain.
- **Deploying ERC-20 tokens** involves writing and deploying a smart contract using tools like **Remix IDE**, and interacting with it through a wallet such as **MetaMask** on the Ethereum network.

ASSESSMENT

Rubrics	Full Mark	Marks Obtained	Remarks
Concept	10		
Planning and Execution/Practical Simulation/ Programming	10		
Result and Interpretation	10		
Record of Applied and Action Learning	10		
Viva	10		
Total	50		

Signature of the Student :

Name :

Signature of the Faculty :

Regn. No. :

Page No.....

*** As applicable according to the experiment.
Two sheets per experiment (10-20) to be used**